

VisionInspect AI - Milestone 4 Final Documentation

1. Project Overview

VisionInspect AI is an industrial manufacturing defect detection and quality inspection system. It combines deep learning computer vision, advanced anomaly detection, semantic segmentation, and automated decision-making to perform real-time quality control on the shop floor.

2. Architecture & ML Pipeline

The core architecture operates via a robust multi-stage AI pipeline:

1. **Category Classification:** ResNet18 identifies the product type (trained on the MVTec AD dataset).
2. **Anomaly Detection:** Deep feature embeddings isolate defective items from normal items using memory banks.
3. **Semantic Segmentation:** U-Net generates sub-pixel masks of the defect regions.
4. **Defect Classification:** Identifies the specific defect subtype (e.g., scratch, crack, contamination).
5. **Secondary Localization:** YOLO11n provides secondary bounding-box localization for discrete objects.
6. **Severity Scoring Engine:** Calculates a 0-100 severity score based on Size (30%), Location (25%), Defect Type (25%), and AI Confidence (20%).

3. Current Quality Decision Logic

The AI automated quality decision logic enforces the following boundaries:

- **Low (0-39):** Minor cosmetic defect $\rightarrow$ **Accept**
- **Medium (40-59):** Moderate quality concern $\rightarrow$ **Reject**
- **High (60-79):** Significant quality issue $\rightarrow$ **Reject**
- **Critical (80-100):** Major structural defect $\rightarrow$ **Reject**

4. Supervisor Review Workflow

- Every inspection automatically enters the **Supervisor Review Queue** (`supervisor_decision = null`).
- Supervisors manually audit the AI's `quality_decision` and formally **Approve** or **Reject** the inspection batch.
- This provides full operational traceability without overriding the underlying AI metrics used for factory yield analytics.

5. API, Database, and Frontend

- **Backend:** FastAPI manages inspection endpoints, RBAC security (Quality Engineers vs. Supervisors), and PostgreSQL queries.
- **Database:** PostgreSQL strictly tracks `quality_decision` (AI decision) vs. `supervisor_decision` (human manual audit).
- **Frontend:** A modern React + Vite dashboard displaying real-time inspection results, severity breakdown dials, visual defect heatmaps, and supervisor review queues.

6. Testing Status & Known Limitations

Extensive automated unit testing, API validation, and UI audits have been successfully completed. **Known Limitations:**

- YOLO11n object detection struggles with subtle pixel-level anomalies (e.g., color shifts, thin cracks). U-Net segmentation serves as the primary ground truth for the severity engine, while YOLO acts as an optional overlay.

7. Deployment Instructions

The system is ready for containerized deployment via Docker.

1. Configure `DATABASE_URL` for PostgreSQL.
2. Build frontend static assets using `npm run build`.
3. Serve backend via Uvicorn: `uvicorn app.main:app --host 0.0.0.0 --port 8000`.